Mansoura University
Faculty of Computers and Information
Department of Information Technology

Database Systems

Lecture 03: Relational Database Design by ER- and EER-to-Relational Mapping

Dr. Nabila Eladawi

OUTLINE

- Enhanced ER (EER) model
- Relational Database Design by ER-to-Relational Mapping
- Relational Database Design by EER-to-Relational Mapping

THE ENHANCED ENTITY-RELATIONSHIP (EER) MODEL

- **Enhanced ER (EER) model**
  - Created to design **more accurate database** schemas
    - Reflect the **data properties** and **constraints more precisely**
  - **More complex requirements** than traditional applications
- **EER model includes all modeling concepts of the ER model**
- In addition, EER includes:
  - **Subclasses** and **superclasses**
  - **Specialization** and **generalization**
  - **Category** or **union type**
  - **Attribute** and **relationship inheritance**

THE ENHANCED ENTITY-RELATIONSHIP (EER) MODEL

- Example: EMPLOYEE may be further grouped into:
- SECRETARY, ENGINEER, TECHNICIAN, …
  - Based on the EMPLOYEE's Job
- MANAGER
  - EMPLOYEEs who are managers
- SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
  - Based on the EMPLOYEE's method of pay

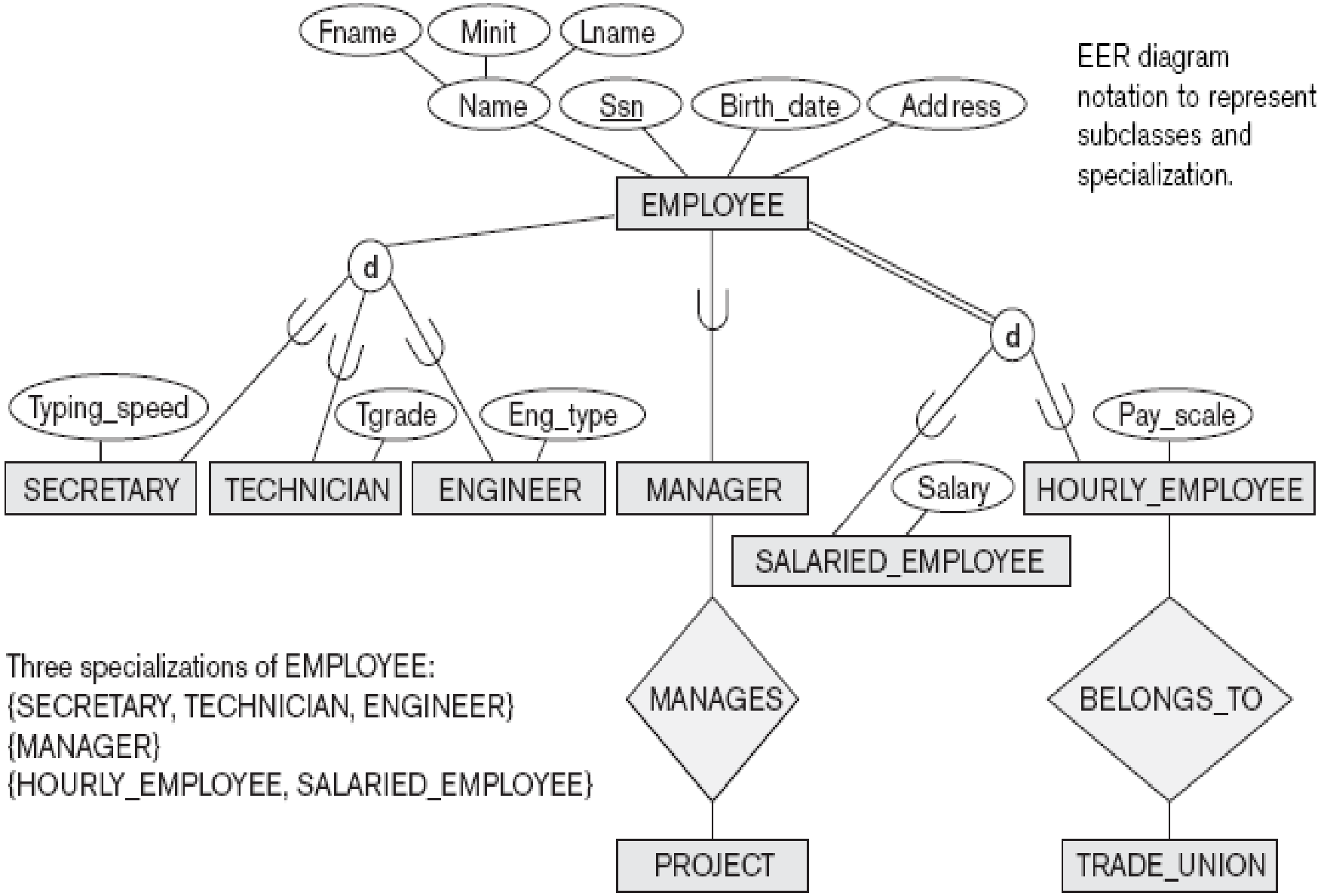
SUBCLASSES, SUPERCLASSES, AND INHERITANCE

- **Superclass:** An entity type that **includes one or more distinct subgroupings of its occurrences**, which require to be represented in a **data model**.
- **Subclass:** A **distinct subgrouping of occurrences** of an entity type, which require to be represented in a **data model**.
- An entity in a subclass represents the same 'real world' object as in the superclass, and **may possess subclass-specific attributes**, as well as those associated with the superclass.
  - We say that an entity that is a member of a **subclass inherits all the attributes** of the entity as a member of the **superclass**. The entity also **inherits all the relationships** in which the superclass participates.

SPECIALIZATION

- **Specialization:** The process of **maximizing the differences between members** of an entity by identifying their **distinguishing characteristics**.
  - It is the process of **defining a set of subclasses of an entity type**; this entity type is called the **superclass** of the specialization.

    Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon job type.
- The specialization process allows us to do the following:
  - Define a **set of subclasses** of an entity type.
  - Establish **additional specific attributes** with each subclass.
  - Establish **additional specific relationship types** between each subclass and other entity types or other subclasses.

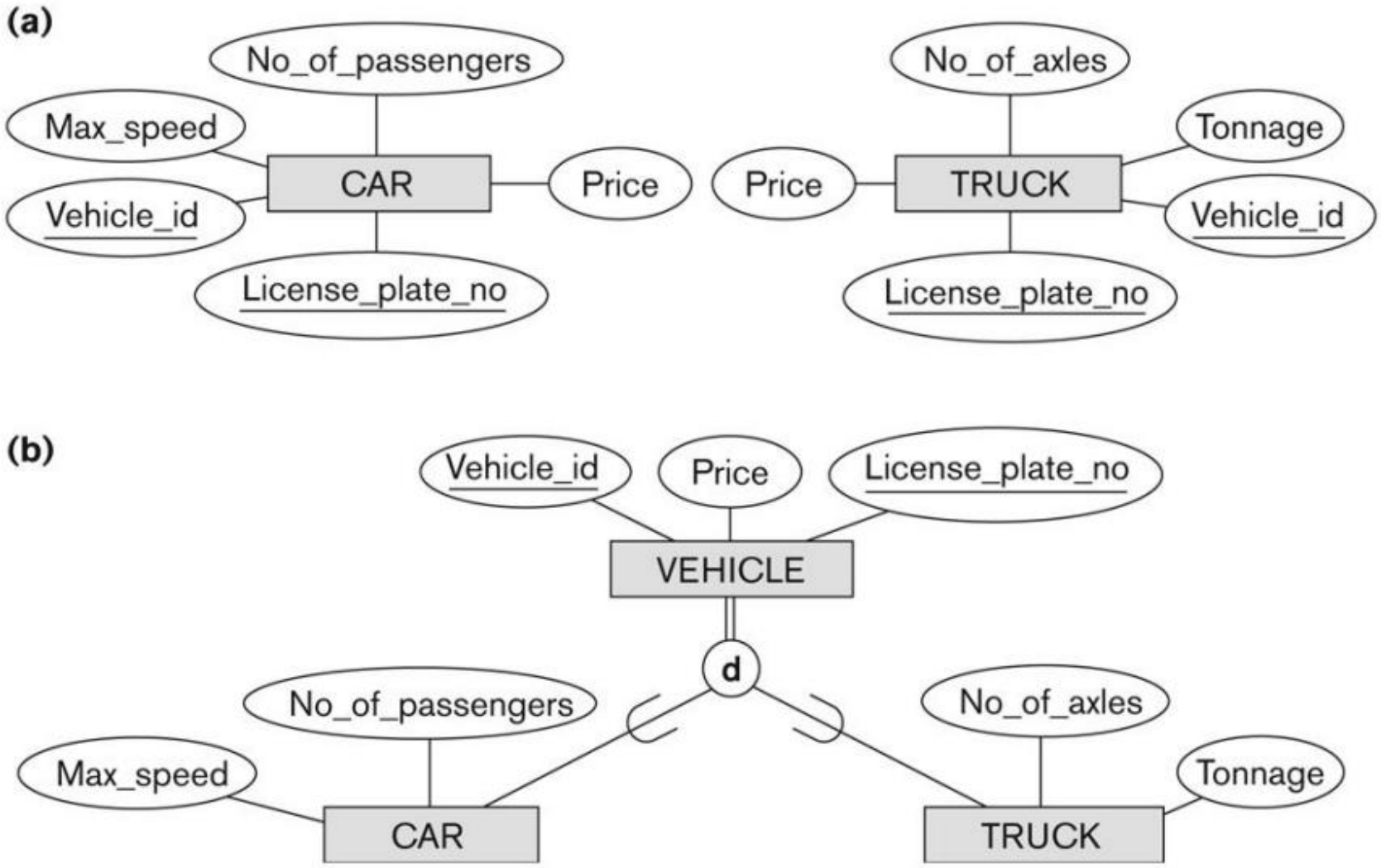


EER diagram notation to represent subclasses and specialization.

Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}

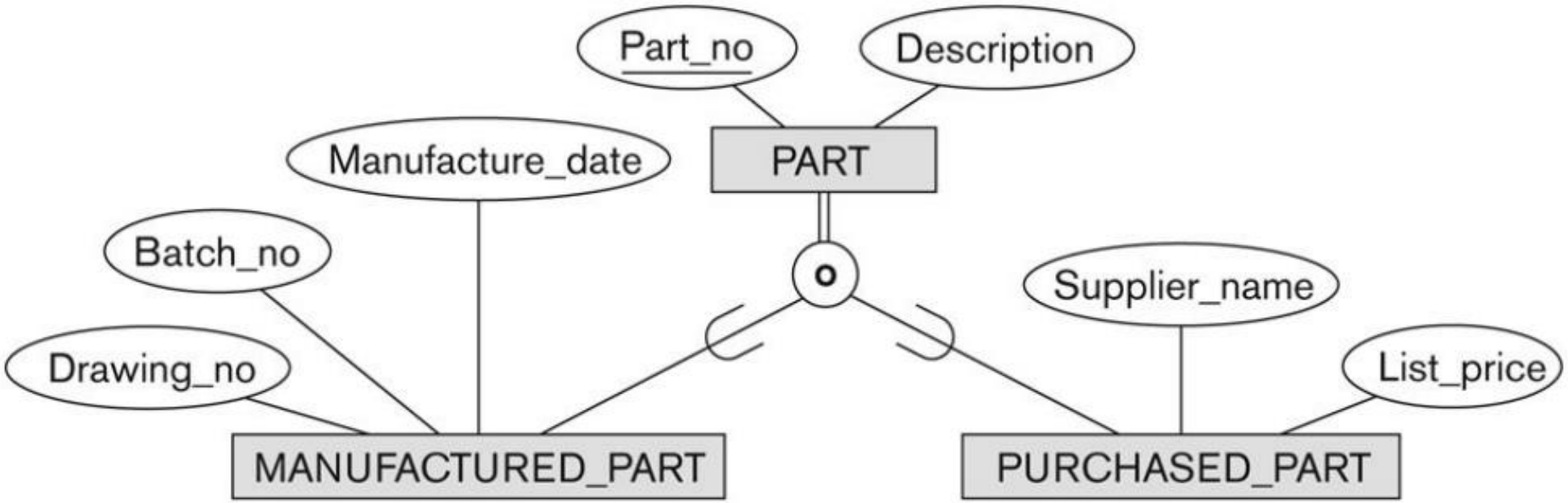
GENERALIZATION

- **Generalization:** The process of **minimizing the differences between entities** by identifying their common characteristics.
  - We suppress the differences among several entity types, identify their common features, and generalize them into a **single superclass** of which the original entity types are special subclasses.
  - Example: CAR, TRUCK generalized into VEHICLE;
    - both CAR, TRUCK become subclasses of the superclass VEHICLE.
  - We can view {CAR, TRUCK} as a specialization of VEHICLE
  - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK


(a)
No_of_passengers
Max_speed
CAR
Price
Vehicle_id
License_plate_no
No_of_axles
Tonnage
Price
TRUCK
Vehicle_id
License_plate_no
(b)
Vehicle_id
Price
License_plate_no
VEHICLE
d
No_of_passengers
Max_speed
CAR
No_of_axles
Tonnage
TRUCK

CONSTRAINTS ON SPECIALIZATION/GENERALIZATION

- **Participation constraint**: Determines whether every member in the superclass must participate as a member of a subclass.
  - A **double line** to connect the superclass to the circle is used to display a **total specialization**.
  - A **single line** is used to display a **partial specialization.**
- **Disjoint constraint**: Describes the relationship between members of the subclasses and indicates whether it is possible for a member of a superclass to be a member of one, or more than one, subclass.
  - The **d** in the circle stands for **disjoint**.
  - The **o** in the circles stands for **overlapping.**


Part_no
Description
Manufacture_date
PART
Batch_no
o
Supplier_name
Drawing_no
List_price
MANUFACTURED_PART
PURCHASED_PART

SPECIALIZATION AND GENERALIZATION HIERARCHIES AND LATTICES

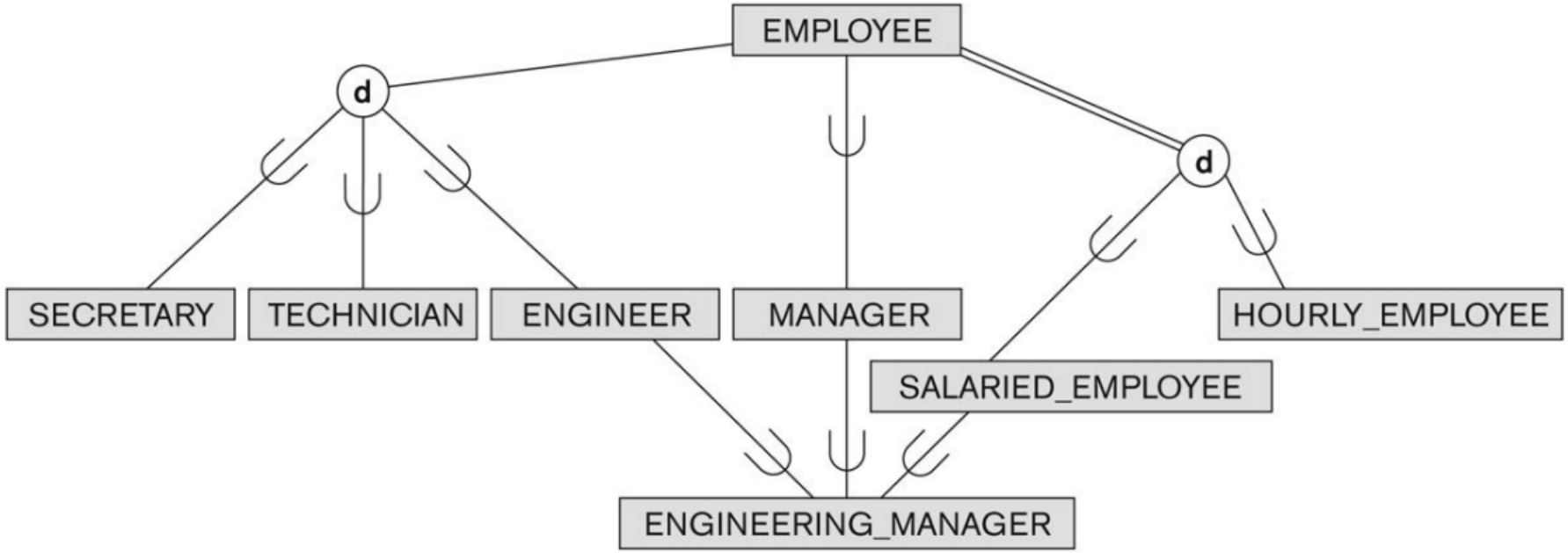
- A subclass itself may have further subclasses specified on it, forming a **hierarchy or a lattice of specializations**.
- A **specialization hierarchy** has the constraint that every subclass participates as a subclass in only one class/subclass relationship; that is, each subclass has only one parent, which results in a tree structure or strict hierarchy.
- In contrast, for a **specialization lattice**, a subclass can be a subclass in more than one class/subclass relationship.

SPECIALIZATION AND GENERALIZATION HIERARCHIES AND LATTICES

- In such a **specialization lattice or hierarchy**, a **subclass inherits** the **attributes** not only of its direct superclass, but also of **all its predecessor super classes** all the way to the root of the hierarchy or lattice if necessary.

- An **entity may exist in several leaf nodes** of the hierarchy, where a **leaf node is a class that has no subclasses of its own**.

- A **subclass** with **more than one superclass is called a shared subclass.**


EMPLOYEE
d
d
SECRETARY
TECHNICIAN
ENGINEER
MANAGER
HOURLY_EMPLOYEE
SALARIED_EMPLOYEE
ENGINEERING_MANAGER

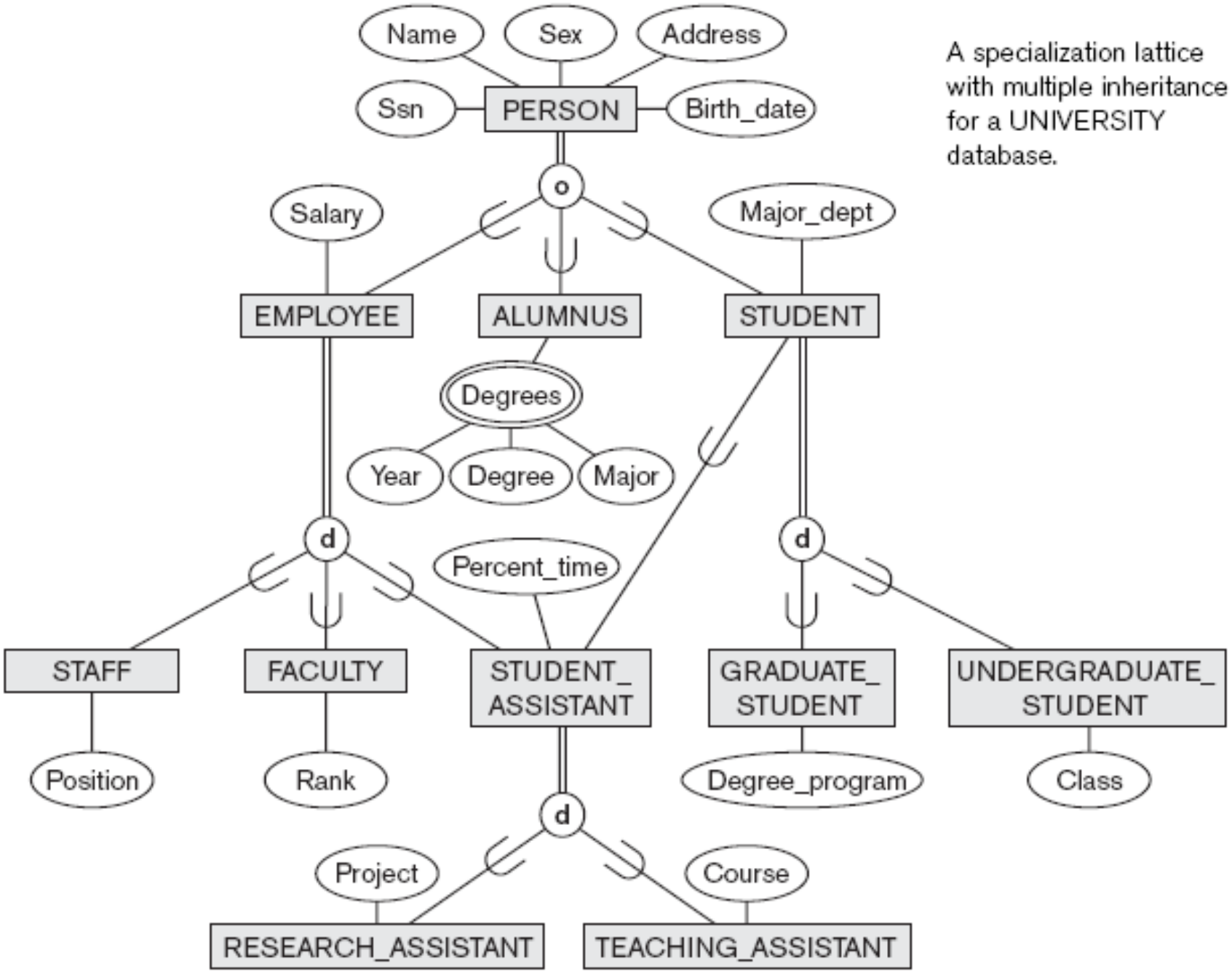
SPECIALIZATION AND GENERALIZATION HIERARCHIES AND LATTICES

- **Multiple inheritance**, where the shared subclass directly **inherits attributes and relationships from multiple classes**.

- Notice that the **existence of at least one shared subclass leads to a lattice** (and hence to **multiple inheritance**); if no shared subclasses existed, we would have a **hierarchy rather than a lattice** and only single inheritance would exist.

EXAMPLE

- 1. The database keeps track of three types of persons: **employees**, **alumni**, and **students**. A person can belong to one, two, or all three of these types. Each person has a **name**, **SSN**, **sex**, **address**, and **birth date**.
- 2. Every **employee** has a **salary**, and there are **three types of employees**: **faculty**, **staff**, and **student assistants**. Each employee belongs to exactly one of these types. For each **alumnus**, a **record of the degree** or degrees that he or she earned at the university is kept, including the **name of the degree**, the **year granted**, and the **major department**. Each student has a **major department**.
- 3. Each **faculty** has a **rank**, whereas each **staff** member has a **staff position**. **Student assistants** are classified further as either **research assistants** or **teaching assistants**, and the **percent of time** that they work is recorded in the database. **Research assistants** have their **research project** stored, whereas **teaching assistants** have the **current course** they work on.
- 4. **Students** are further classified as either **graduate** or **undergraduate**, with the specific attributes **degree program** (M.S., Ph.D., M.B.A., and so on) for **graduate students** and **class** (freshman, sophomore, and so on) for **undergraduates**.



A specialization lattice with multiple inheritance for a UNIVERSITY database.

UTILIZING SPECIALIZATION AND GENERALIZATION IN REFINING CONCEPTUAL SCHEMAS

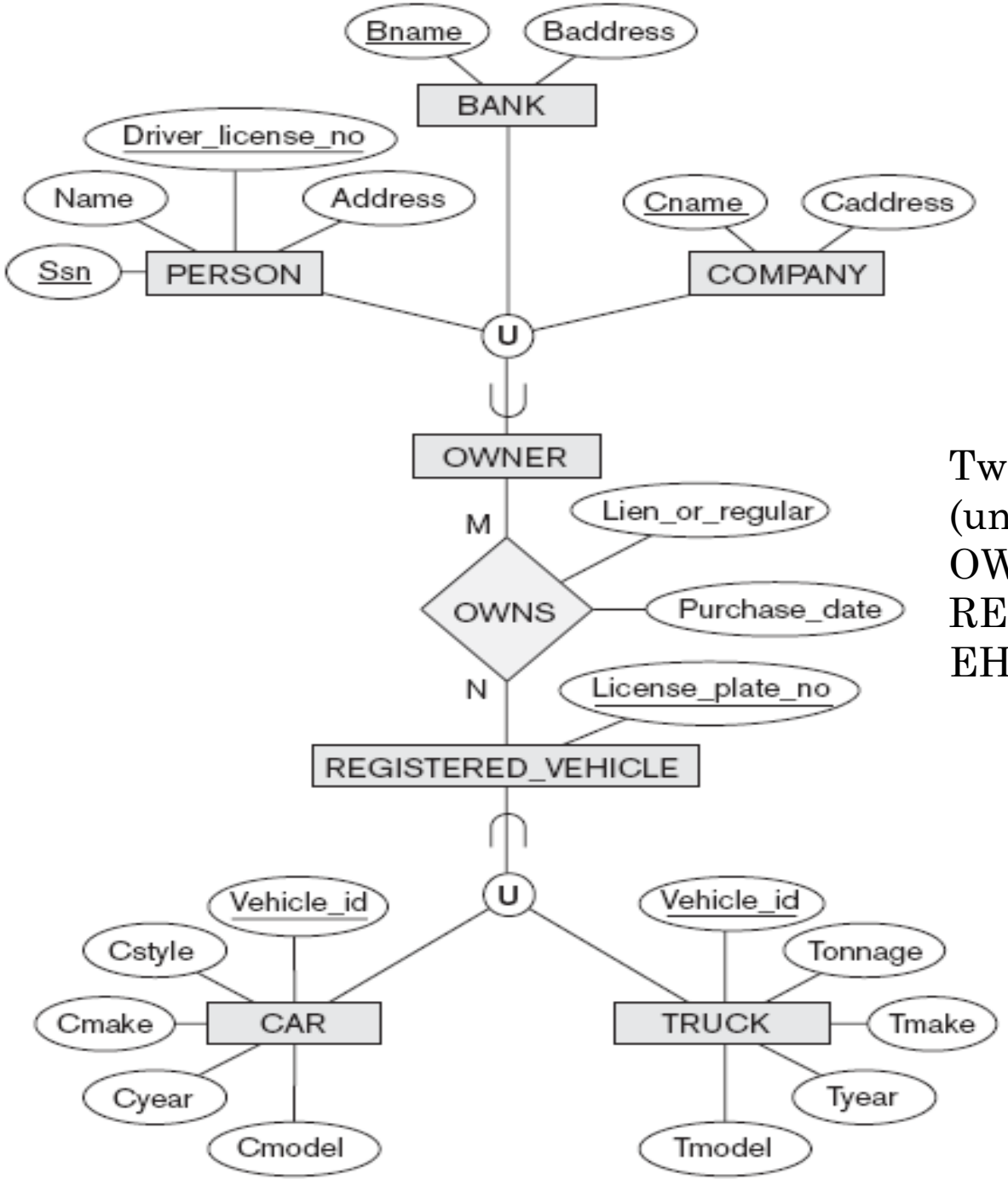
- **Specialization process**
  - Start with entity type then define subclasses by successive specialization.
  - Top-down conceptual refinement process.

- **Bottom-up conceptual synthesis**
  - Involves generalization rather than specialization.

MODELING OF UNION TYPES USING CATEGORIES

- **Union type or a category:**
  - It represents a **single superclass/subclass relationship** with **more than one superclass.**
  - **Subclass** represents a **collection of objects** that is a **subset of the UNION of distinct entity types.**
  - **Attribute inheritance** works more **selectively.**
  - **Category** can be **total** or **partial.**
  - **Difference from shared subclass, which is a:**
    - **shared subclass member must exist in all of its superclasses**

- **Some modeling methodologies do not have union types**



Two categories (union types): OWNER and REGISTERED_VEHICLE.

DESIGN CHOICES FOR SPECIALIZATION/GENERALIZATION

- Many **specializations and subclasses can be defined** to make the **conceptual model** accurate.
- If **subclass** has few **specific attributes** and no **specific relationships**
  - Can be **merged** into the **superclass**.
- If all the subclasses of a specialization/generalization have **few specific attributes** and no specific relationships.
  - Can be merged into the superclass.
  - Replace with one or more type attributes that specify the subclass or subclasses that each entity belongs to.

DESIGN CHOICES FOR SPECIALIZATION/GENERALIZATION (CONT'D.)

- **Union types and categories** should generally be **avoided**.
- Choice of **disjoint/overlapping** and **total/partial constraints** on specialization/generalization
  - Driven by rules in **miniworld being modeled**

OUTLINE

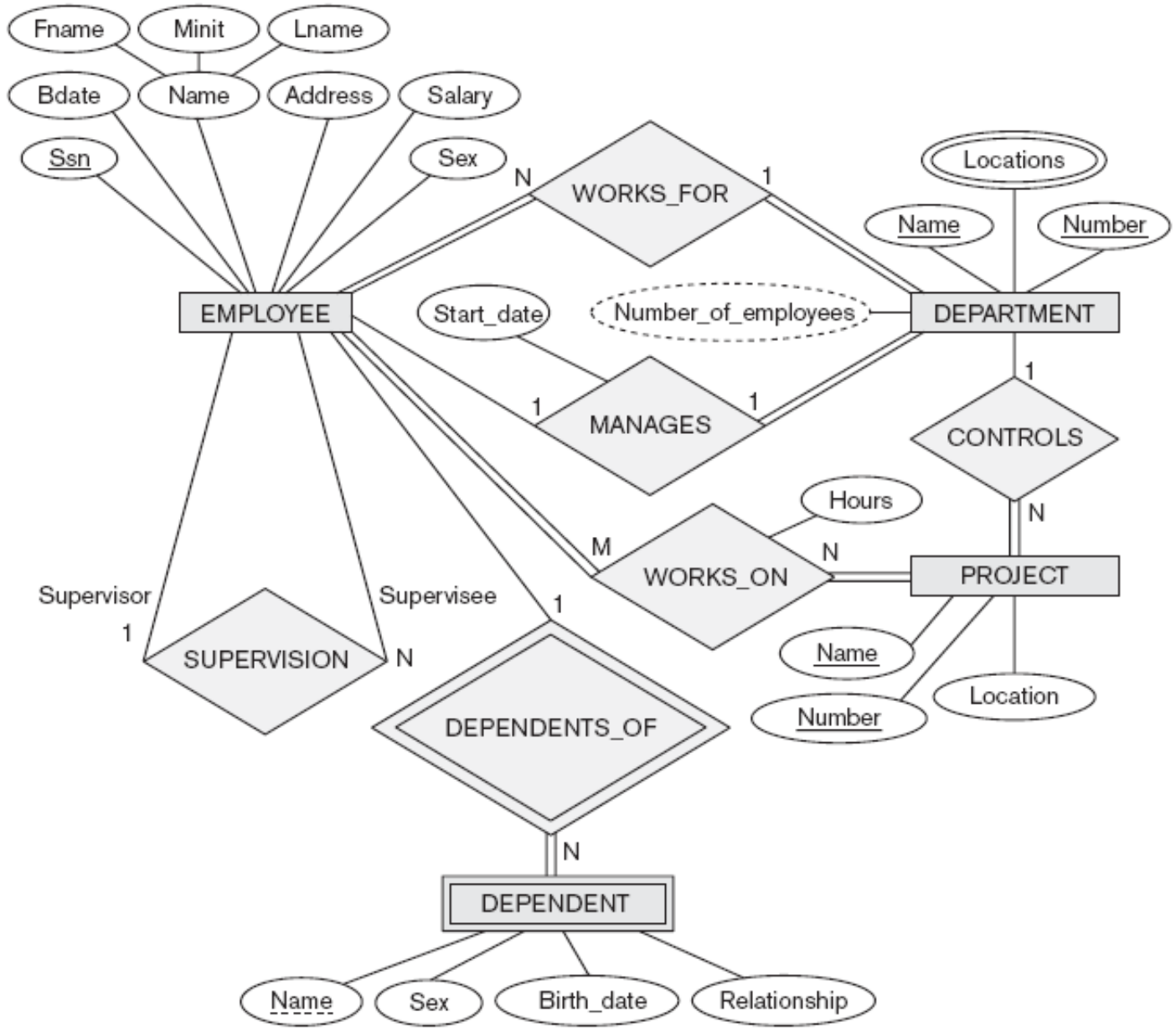
- Enhanced ER (EER) model
- **Relational Database Design by ER-to-Relational Mapping**
- Relational Database Design by EER-to-Relational Mapping

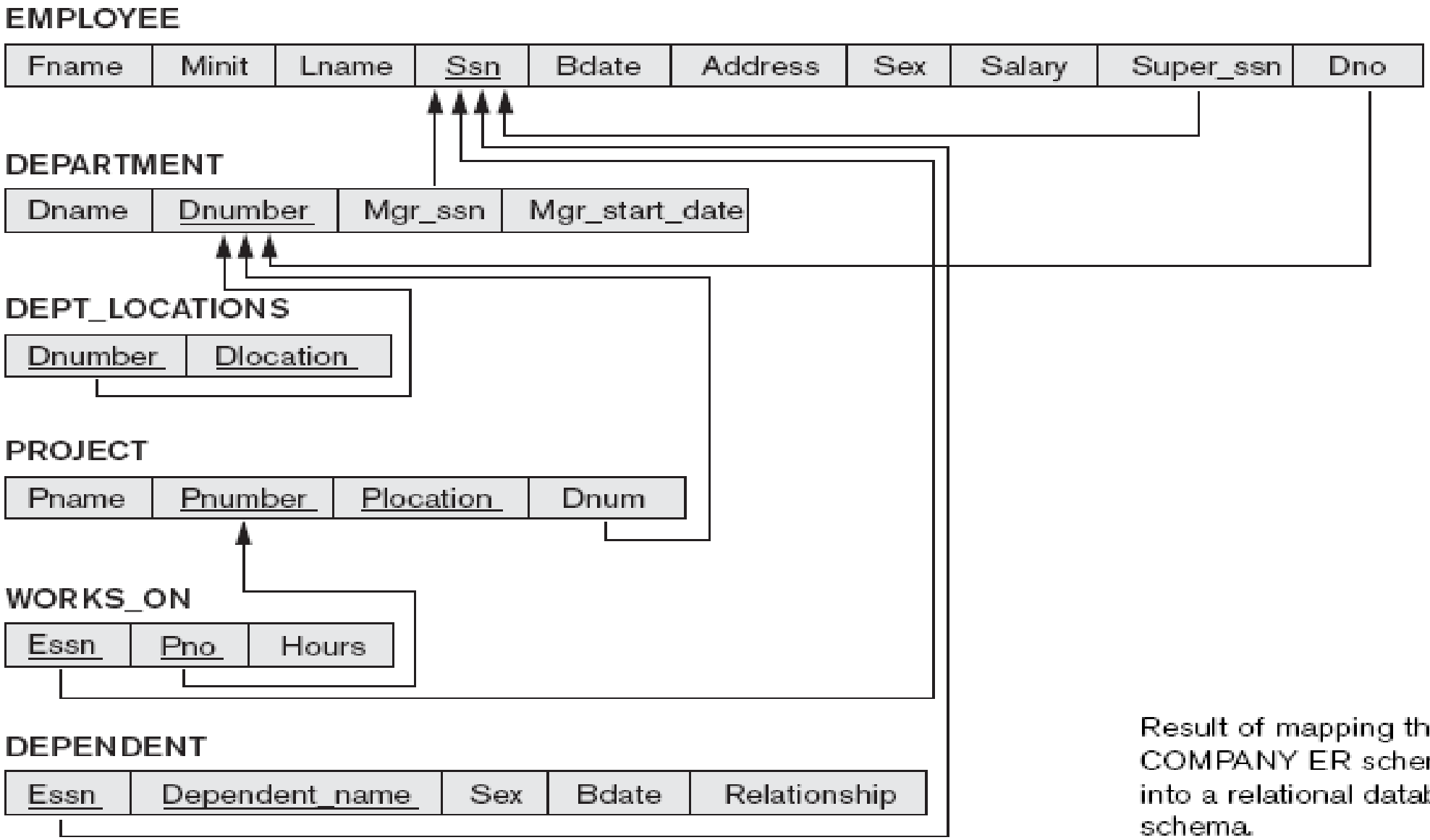
RELATIONAL DATABASE DESIGN BY ER- AND EER-TO-RELATIONAL MAPPING

- Design a **relational database schema** based on a **conceptual schema design**.
- **Seven-step algorithm** to convert the basic **ER model constructs into relations**.
- **Additional steps** for **EER model**.

RELATIONAL DATABASE DESIGN USING ER-TO-RELATIONAL MAPPING

The ER conceptual schema diagram for the COMPANY database.





Result of mapping the COMPANY ER schema into a relational database schema.

ER-TO-RELATIONAL MAPPING ALGORITHM

- COMPANY database example
  - Assume that the **mapping** will **create tables** with simple **single-valued attributes**
- **Step 1: Mapping of Regular Entity Types**
  - For each regular entity type, **create** a **relation** R that includes all the **simple attributes** of E
  - Include only the **simple component attributes** of a composite attribute.
  - Choose one of the key attributes of E as the **PK** for R.
  - Called **entity relations**
    - Each tuple represents an **entity instance**

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary

DEPARTMENT

Dname	Dnumber

PROJECT

Pname	Pnumber	Plocation

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- **Step 2: Mapping of Weak Entity Types**
  - For each weak entity type, **create a relation R and include all simple attributes** of the entity type as attributes of R.
  - Include **primary key attribute of owner** as **foreign key** attributes of R.
  - The **primary key of R is the combination of the primary key(s)** of the owner(s) and the partial key of the weak entity type W, if any.

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- Step 3: **Mapping of Binary 1:1 Relationship Types**
  - For each binary 1:1 relationship type
    - **Identify relations** that correspond to entity types participating in R
  - Possible approaches:
    - **Foreign key approach**
      - Choose one of the relations—S, say—and include as a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S.
    - **Merged relationship approach**
      - Merge the two entity types and the relationship into a single relation. This is possible when both participations are total
    - **Cross-reference or relationship relation approach**
      - set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types.

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- **Step 4: Mapping of Binary 1:N Relationship Types**
  - For each regular binary 1:N relationship type:
    - Identify relation that represents participating entity type at **N-side of relationship type.**
    - Include **primary key of other entity type as foreign key in S.**
    - Include **simple attributes of 1:N relationship** type as **attributes of S.**
  - Alternative approach:
    - Use the relationship relation (cross-reference) option.
    - The primary key of R is the same as the primary key of S. This option can be used if few tuples in S participate in the relationship to **avoid excessive NULL values in the foreign key**.

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- **Step 5: Mapping of Binary M:N Relationship Types**
  - For each binary M:N relationship type
    - Create a new relation R
    - Include primary key of participating entity types as foreign key attributes in R
    - Include any simple attributes of M:N relationship type

WORKS_ON

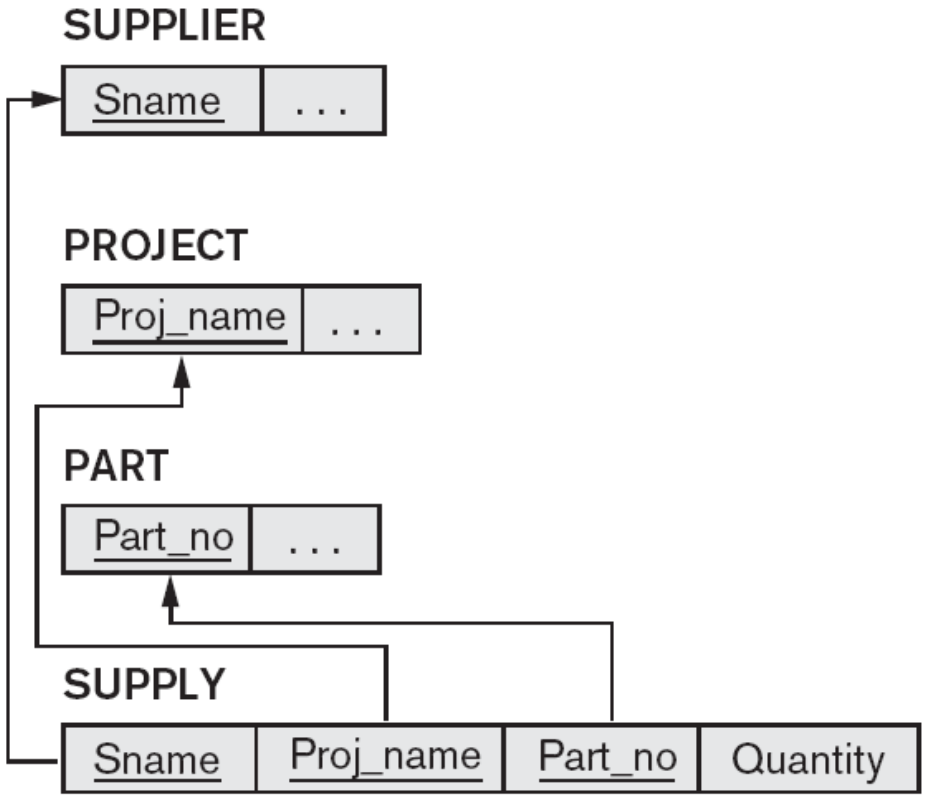
Essn	Pno	Hours

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- **Step 6: Mapping of Multivalued Attributes**
  - For each multivalued attribute A, Create a new relation R include:
    - An attribute corresponding to A,
    - The primary key attribute K—as a foreign key in R
    - Primary key of R is the combination of A and K
    - If the multivalued attribute is composite, include its simple components

ER-TO-RELATIONAL MAPPING ALGORITHM (CONT'D.)

- Step 7: Mapping of *N*-ary Relationship Types
- For each n-ary relationship type R
  - Create a new relation S to represent R
  - Include primary keys of participating entity types as foreign keys
  - Include any simple attributes as attributes
  - The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity types.



OUTLINE

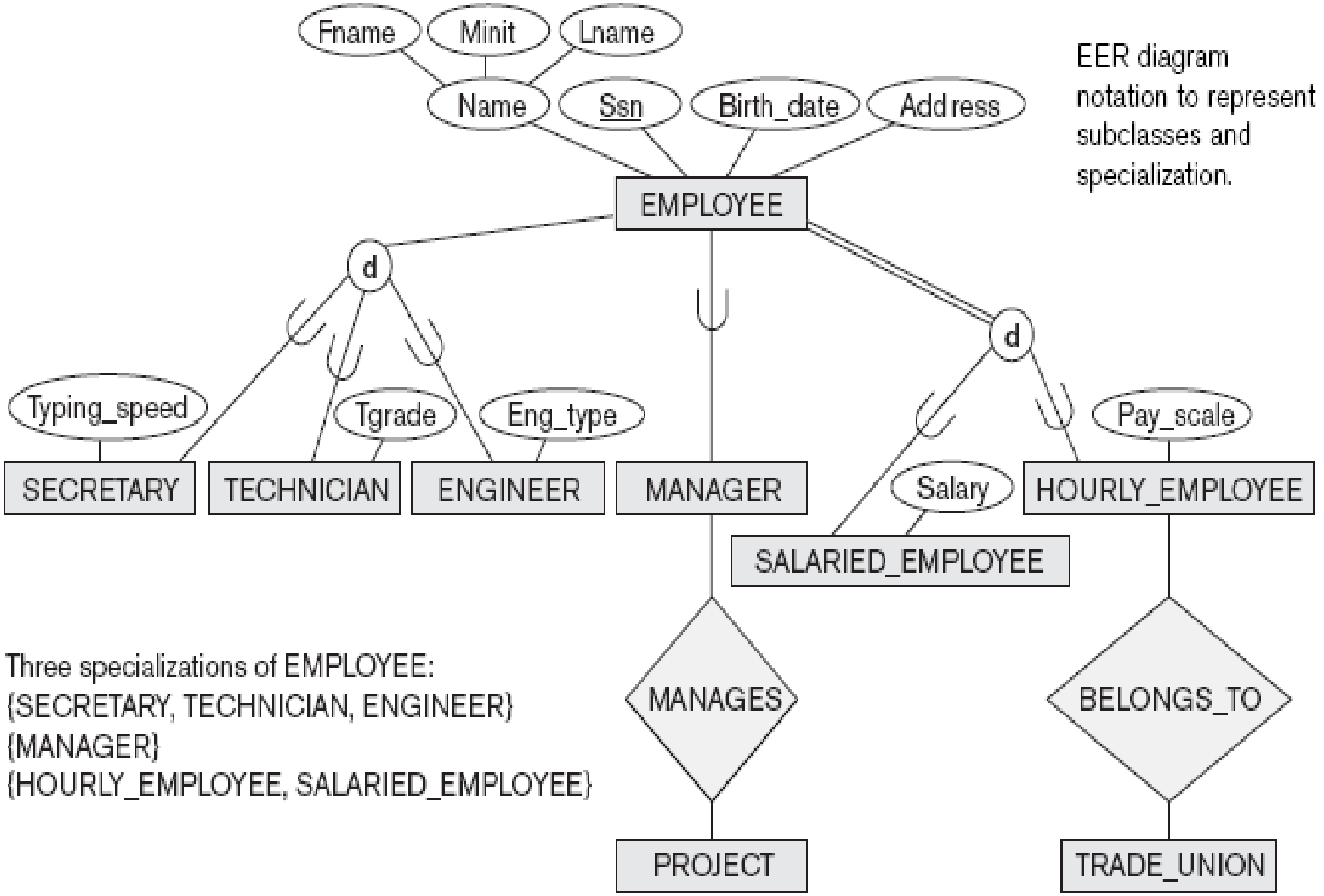
- Enhanced ER (EER) model
- Relational Database Design by ER-to-Relational Mapping
- **Relational Database Design by EER-to-Relational Mapping**

MAPPING OF SPECIALIZATION OR GENERALIZATION

- **Step 8: Options for Mapping Specialization or Generalization**
  - Option 8A: **Multiple relations—superclass and subclasses**
    - For any specialization (total or partial, disjoint or overlapping) **creates a relation L for the superclass C and its attributes**, plus a **relation Li for each subclass Si**;
    - each Li includes the specific (or local) attributes of Si, plus the primary key of the superclass C, which is propagated to Li and becomes its primary key. It also becomes a foreign key to the superclass relation.
  - Option 8B: **Multiple relations—subclass relations only**
    - Subclasses are total
    - Specialization has disjointedness constraint
    - Create a relation Li for each subclass Si, $1 \leq i \leq m$, with the attributes Attrs(Li) = {attributes of Si} $\cup$ {k, a1, ..., an} and PK(Li) = k.

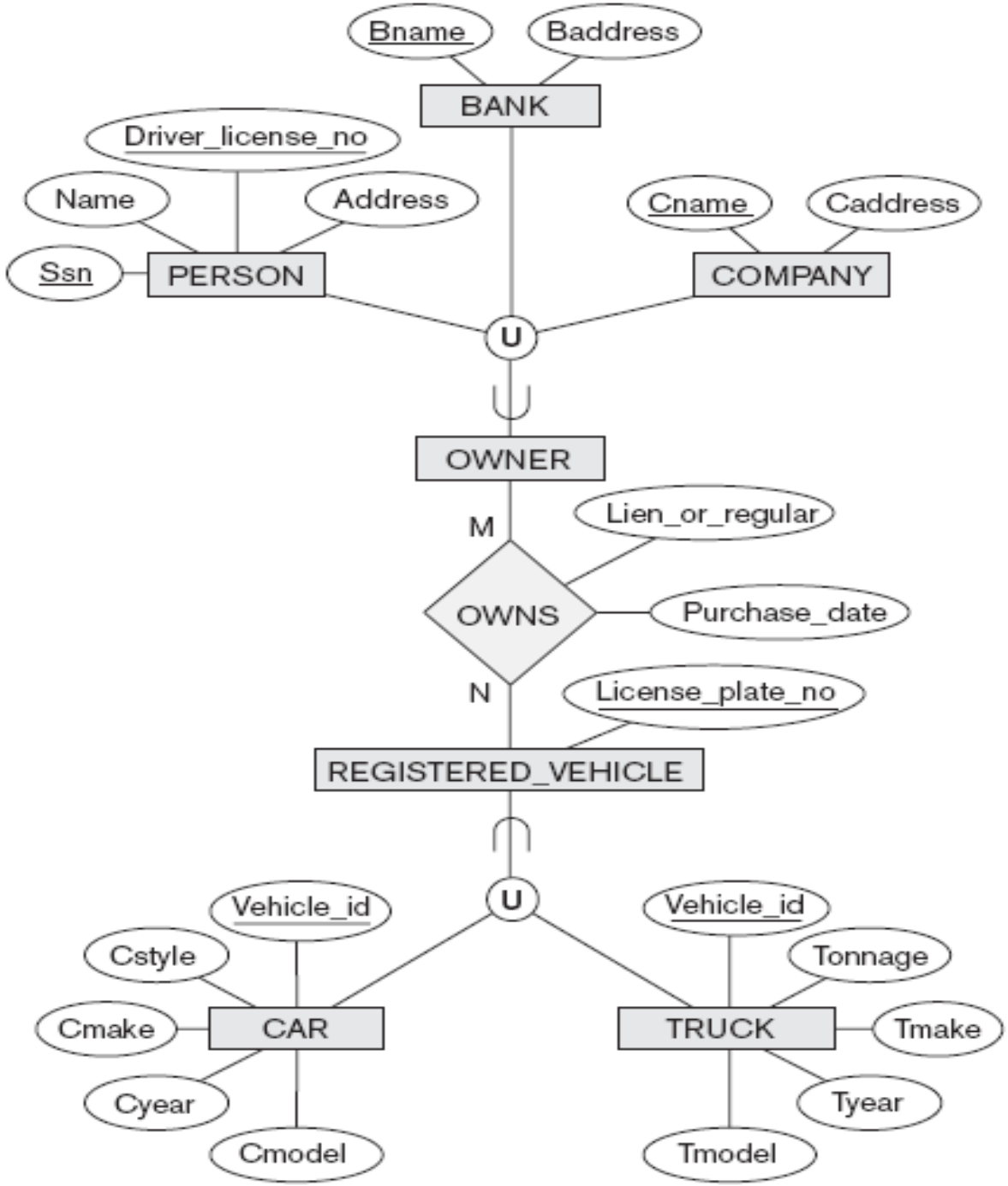
MAPPING OF SPECIALIZATION OR GENERALIZATION (CONT'D.)

- Option 8C: **Single relation with one type attribute**
  - Type or discriminating attribute indicates subclass of tuple
  - Subclasses are disjoint
    - Potential for generating many NULL values if many specific attributes exist in the subclasses
- Option 8D: **Single relation with multiple type attributes**
  - Subclasses are overlapping
  - Will also work for a disjoint specialization



EER diagram notation to represent subclasses and specialization.

Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}



Two categories (union types): OWNER and REGISTERED_VEHICLE.

(a) EMPLOYEE

Ssn	Fname	Minit	Lname	Birth_date	Address	Job_type

SECRETARY

Ssn	Typing_speed

TECHNICIAN

Ssn	Tgrade

ENGINEER

Ssn	Eng_type

(b) CAR

Vehicle_id	License_plate_no	Price	Max_speed	No_of_passengers

TRUCK

Vehicle_id	License_plate_no	Price	No_of_axles	Tonnage

(c) EMPLOYEE

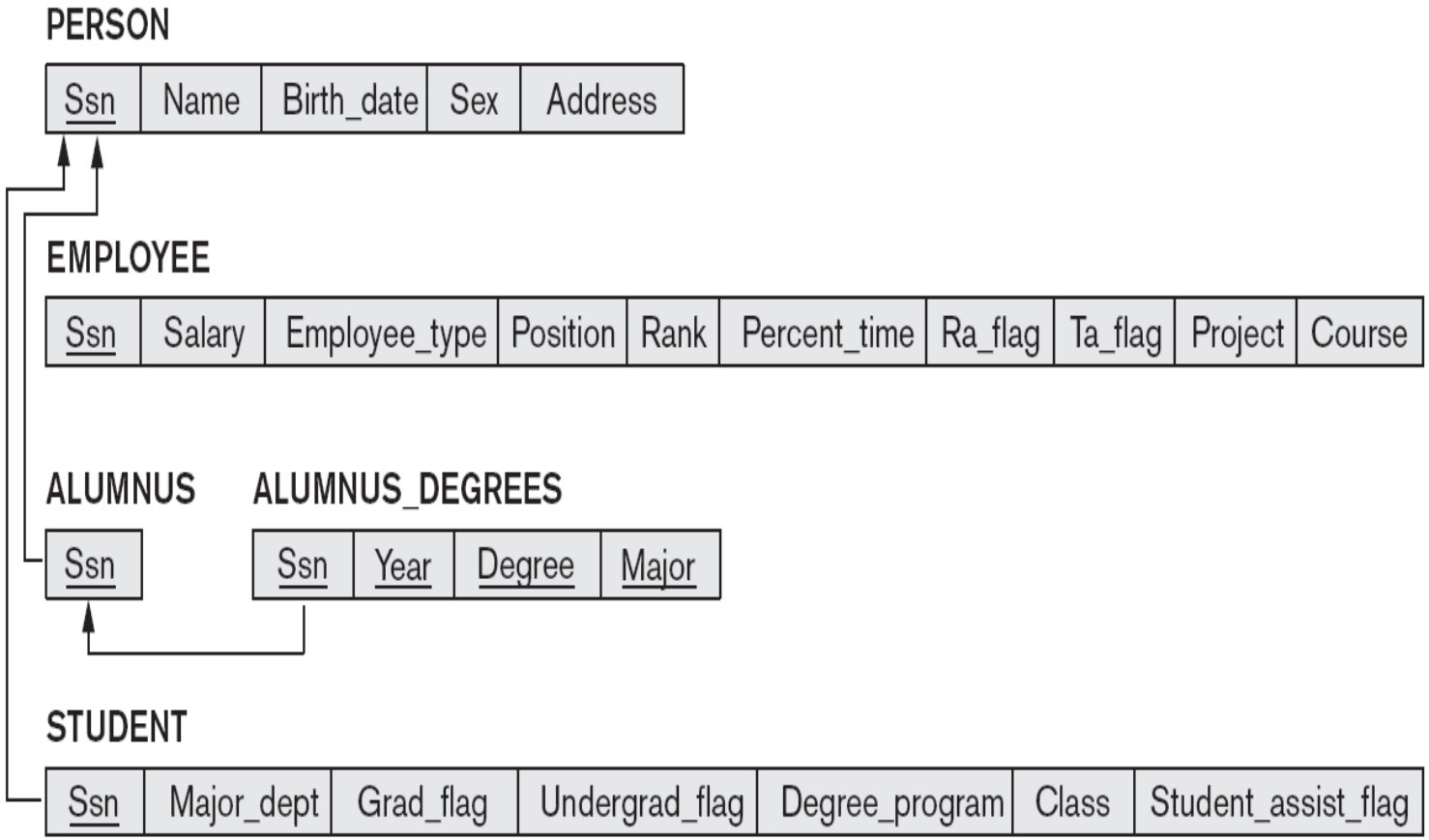
Ssn	Fname	Minit	Lname	Birth_date	Address	Job_type	Typing_speed	Tgrade	Eng_type

(d) PART

Part_no	Description	Mflag	Drawing_no	Manufacture_date	Batch_no	Pflag	Supplier_name	List_price

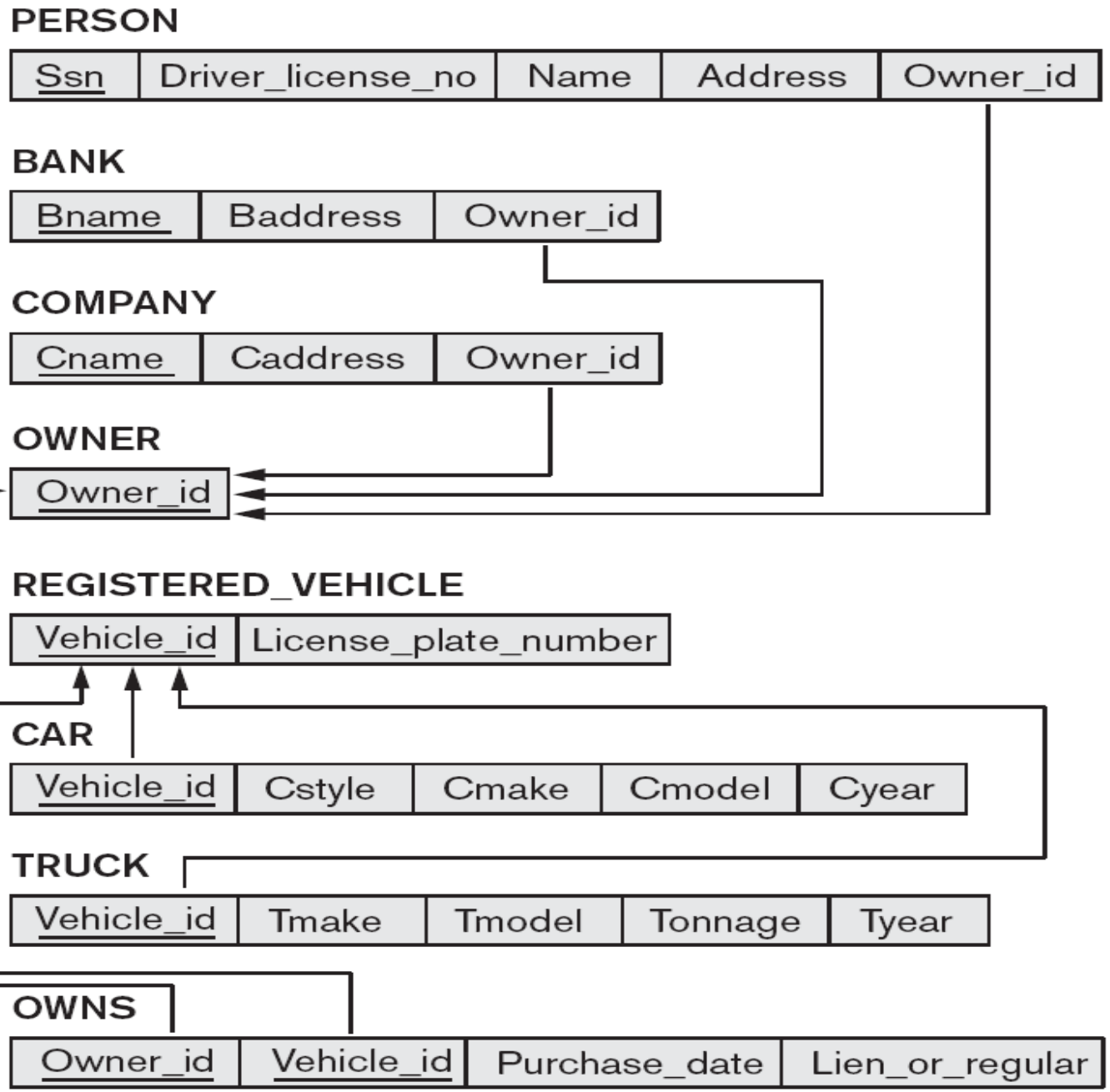
MAPPING OF SHARED SUBCLASSES (MULTIPLE INHERITANCE)

- Apply any of the options discussed in step 8 to a shared subclass



MAPPING OF CATEGORIES (UNION TYPES)

- Step 9: **Mapping of Union Types (Categories)**
  - Defining super classes have different keys
  - Specify a new key attribute
    - Surrogate key


PERSON
Ssn
Driver_license_no
Name
Address
Owner_id
BANK
Bname
Baddress
Owner_id
COMPANY
Cname
Caddress
Owner_id
OWNER
Owner_id
REGISTERED_VEHICLE
Vehicle_id
License_plate_number
CAR
Vehicle_id
Cstyle
Cmake
Cmodel
Cyear
TRUCK
Vehicle_id
Tmake
Tmodel
Tonnage
Tyear
OWNS
Owner_id
Vehicle_id
Purchase_date
Lien_or_regular

Questions?